Systemy operacyjne

Lista zadań nr 3

Na zajęcia 27–29 października 2026

Należy przygotować się do zajęć czytając następujące rozdziały książek:

- Linux Device Drivers[1] [2], Jonathan Corbet, Alessandro Rubini, Greg Kroah-Hartman, wydanie trzecie, przedziały stron: [5, 8], [43, 49], [397, 407]
- Essential Linux Device Drivers, Sreekrishnan Venkateswaran, 2008: rozdział 4, podsekcje "Udev" i "Hotplug and Coldplug"

UWAGA! Każdy student na potrzeby prezentacji zadania ma uruchomić program emulatora terminala w trybie pełnoekranowym. Należy wybrać dużą czcionkę, kontrastowe kolory i jasne tło. Bądź uprzejmy wobec osób, które są oddalone od ekranu!

Zadanie 1. Na podstawie kodu źródłowy sinit.c[3] opowiedz jakie zadania pełni minimalny program rozruchowy `sinit`. Jakie akcje wykonuje pod wpływem wysyłania do niego sygnałów wymienionych w tablicy «`sigmap`»? Do czego służą procedury `sigprocmask(2)` i `sigwait(3)`? W jaki sposób grzebie swoje dzieci?

Zadanie 2. Rozpoczynając od definicji wywołania systemowego kill[4] prześledź w jaki sposób Linuks dostarcza sygnał do jednego konkretnego procesu (nie grupy). Zwróć uwagę na to, by nie omawiać mało istotnych (w kontekście dostarczania sygnału) wywołań.

Omów znaczenie pól: `sighand`, `blocked`, `pending` znajdujących się w strukturze task_struct[5].

Zadanie 3. Funkcja exit_to_user_mode_loop[6] powoduje wykonanie oczekujących operacji przed powrotem do przestrzeni użytkownika. Jedną z takich operacji jest obsługa sygnałów arch_do_signal_or_restart[7]. Pokaż, w jaki sposób jądro przygotowuje obsługę sygnału, na podstawie wybranej przez Ciebie architektury.[8] W tym celu przejdź przez funkcje `handle_signal` i `setup_rt_frame`. Zwróć uwagę na to, by nie omawiać mało istotnych wywołań.

Wskazówka: VDSO to funkcje jądra, zmapowane do przestrzeni użytkownika, które proces może wołać bez konieczności zmiany kontekstu.

Wskazówka 2: Dopisek `__user` przy typie obiektu oznacza, że obiekt ten znajduje się w przesztrzeni użytkownika. Powoduje to równocześnie wymuszenie dodatkowych sprawdzeń/zabezpieczeń podczas wykorzystywania tego obiektu w jądrze.

Ściągnij ze strony przedmiotu archiwum «`so26_lista_3.tar.gz`», następnie rozpakuj i zapoznaj się z dostarczonymi plikami.

UWAGA! Można modyfikować tylko te fragmenty programów, które zostały oznaczone w komentarzu napisem «`TODO`».

Zadanie 4. Uzupełnij program «`cycle.c`», w którym procesy grają w piłkę przy pomocy sygnału «`SIGUSR1`». Proces główny tworzy n dzieci. Każde z nich czeka na piłkę, a po jej odebraniu podaje ją do swojego starszego brata. Zauważ, że najstarszy brat nie zna swojego najmłodszego rodzeństwa, ale zna je ojciec – więc należy go wciągnąć do gry! Niech tata rozpocznie grę rzucając piłkę do najmłodszego dziecka. Kiedy znudzi Ci się obserwowanie procesów grających w piłkę możesz nacisnąć «`CTRL+C`» co wyśle «`SIGINT`» do całej rodziny. Możesz wprowadź do zabawy dodatkową piłkę wysyłając sygnał «`SIGUSR1`» poleceniem «`kill`». Czy piłki ostatecznie skleją się w jedną? W rozwiązaniu należy wykorzystać `sigprocmask(2)`, `sigsuspend(2)` i `kill(2)`.

UWAGA! Użycie funkcji `sleep(3)` lub podobnych do właściwego uszeregowania procesów jest zabronione!

[1] `https://lwn.net/Kernel/LDD3/`
[2] w skrócie: LDD
[3] `https://git.suckless.org/sinit/files.html`
[4] `https://elixir.bootlin.com/linux/v7.2.11/source/kernel/signal.c#L4030`
[5] `https://elixir.bootlin.com/linux/v7.2.11/source/include/linux/sched.h#L826`
[6] `https://elixir.bootlin.com/linux/v7.2.11/source/kernel/entry/common.c#L97`
[7] `https://elixir.bootlin.com/linux/v7.2.11/source/arch/riscv/kernel/signal.c#L499`
[8] Kod dla RISC-V jest trochę prostszy niż dla x86, ale x86 było na ASK-u.

Zadanie 5. Procedury `setjmp(3)` i `longjmp(3)` z biblioteki standardowej języka `C` służą do wykonywania nielokalnych skoków. Uproszczone odpowiedniki tych procedur znajdują się w pliku «`libcsapp/Setjmp.s`», a definicja «`Jmpbuf`» w pliku «`include/csapp.h`». Wyjaśnij co robią te procedury, a następnie przeprowadź uczestników zajęć przez ich kod. Dlaczego «`Jmpbuf`» nie przechowuje wszystkich rejestrów procesora? Czemu «`Longjmp`» zapisuje na stos wartość przed wykonaniem instrukcji «`ret`»?

Zadanie 6. Uzupełnij program «`game`» tj. prostą grę w szybkie obliczanie sumy dwóch liczb. Zadaniem procedury «`readnum`» jest wczytać od użytkownika liczbę. Jeśli w międzyczasie przyjdzie sygnał, to procedura ma natychmiast wrócić podając numer sygnału, który przerwał jej działanie. W przeciwnym przypadku zwraca zero i przekazuje wczytaną liczbę przez pamięć pod wskaźnikiem «`num_p`». Twoja implementacja procedury «`readnum`» musi wczytać cały wiersz w jednym kroku. Należy wykorzystać procedury `siglongjmp(3)`, `sigsetjmp(3)` i `alarm(2)`. Pamiętaj, żeby po wczytaniu ciągu znaków zakończyć go znakiem `NUL` i wyłączyć czasomierz! Kiedy Twój program będzie zachowywać się poprawnie zamień procedury **nielokalnych skoków** na `longjmp(3)` i `setjmp(3)`. Czemu program przestał działać?

UWAGA! We FreeBSD i macOS zamiast «`longjmp`» i «`setjmp`» należy użyć odpowiednio «`_longjmp`» i «`_setjmp`».

Zadanie 7. Program «`coro`» wykonuje trzy współprogramy[9] połączone ze sobą w potok bez użycia `pipe(2)`. Pierwszy z nich czyta ze standardowego wejścia znaki, kompresuje białe znaki i zlicza słowa. Drugi usuwa wszystkie znaki niebędące literami. Trzeci zmienia wielkość liter i drukuje znaki na standardowe wyjście.

W wyniku wykonania procedury «`coro_yield`» współprogram przekazuje niezerową liczbę do następnego współprogramu, który otrzyma tę wartość w wyniku powrotu z «`coro_yield`». Efektywnie procedura ta implementuje **zmianę kontekstu**. Taką prymitywną formę **wielozadaniowości kooperacyjnej** (ang. *cooperative multitasking*) można zaprogramować za pomocą `setjmp(3)` i `longjmp(3)`.

Zaprogramuj procedurę «`coro_switch`» tak, by wybierała następny współprogram do uruchomienia i przełączała na niego kontekst. Jeśli współprogram przekazał wartość parametru «`EOF`», to należy go usunąć z listy aktywnych współprogramów.

Program używa listy dwukierunkowej «`TAILQ`» opisanej w `queue(3)`. Zmienna «`runqueue`» przechowuje listę aktywnych współprogramów, «`running`» bieżąco wykonywany współprogram, a «`dispatcher`» kontekst programu, do którego należy wrócić, po zakończeniu wykonywania ostatniego aktywnego współprogramu.

[9] `https://en.wikipedia.org/wiki/Coroutine`